Combinational Circuit Design

Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



Combinational Circuit

a circuit whose output depends only on its current input

To design a circuit, we always follow the same three steps.

(1) **truth table**

write the desired output for every input

(2) **Boolean expression**

turn the table into an expression, then simplify it

(3) **gates**

translate the expression into a circuit

(1) truth table

write the desired output for every input

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

(2) Boolean expression

turn the table into an expression, then simplify it

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

becomes

$$y = \overline{a}\,\overline{b}\,\overline{c} + \overline{a}\,\overline{b}\,c + a\,\overline{b}\,\overline{c} + a\,\overline{b}\,c + a\,b\,c$$

To simplify the notation, we define a **minterm** to be the AND of every input (each in true or negated form), for some row of the truth table.

Reading each row as a binary number gives a name (from m_0 to m_{2^n-1}).

a	b	c	name	minterm
0	0	0	m_0	$\overline{a}\,\overline{b}\,\overline{c}$
0	0	1	m_1	$\overline{a}\,\overline{b}\,c$
0	1	0	m_2	$\overline{a}\,b\,\overline{c}$
0	1	1	m_3	$\overline{a}\,b\,c$
1	0	0	m_4	$a\,\overline{b}\,\overline{c}$
1	0	1	m_5	$a\,\overline{b}\,c$
1	1	0	m_6	$a\,b\,\overline{c}$
1	1	1	m_7	$a\,b\,c$

$$y = \overline{a}\,\overline{b}\,\overline{c} + \overline{a}\,\overline{b}\,c + a\,\overline{b}\,\overline{c} + a\,\overline{b}\,c + a\,b\,c$$

becomes

$$y = m_0 + m_1 + m_4 + m_5 + m_7$$

Alternatively, we can define a **maxterm** to be the OR of every input (each in true or negated form), for some row of the truth table.

Reading each row as a binary number gives a name (from M_0 to M_{2^n-1}).

a	b	c	name	maxterm
0	0	0	M_0	$a + b + c$
0	0	1	M_1	$a + b + \overline{c}$
0	1	0	M_2	$a + \overline{b} + c$
0	1	1	M_3	$a + \overline{b} + \overline{c}$
1	0	0	M_4	$\overline{a} + b + c$
1	0	1	M_5	$\overline{a} + b + \overline{c}$
1	1	0	M_6	$\overline{a} + \overline{b} + c$
1	1	1	M_7	$\overline{a} + \overline{b} + \overline{c}$

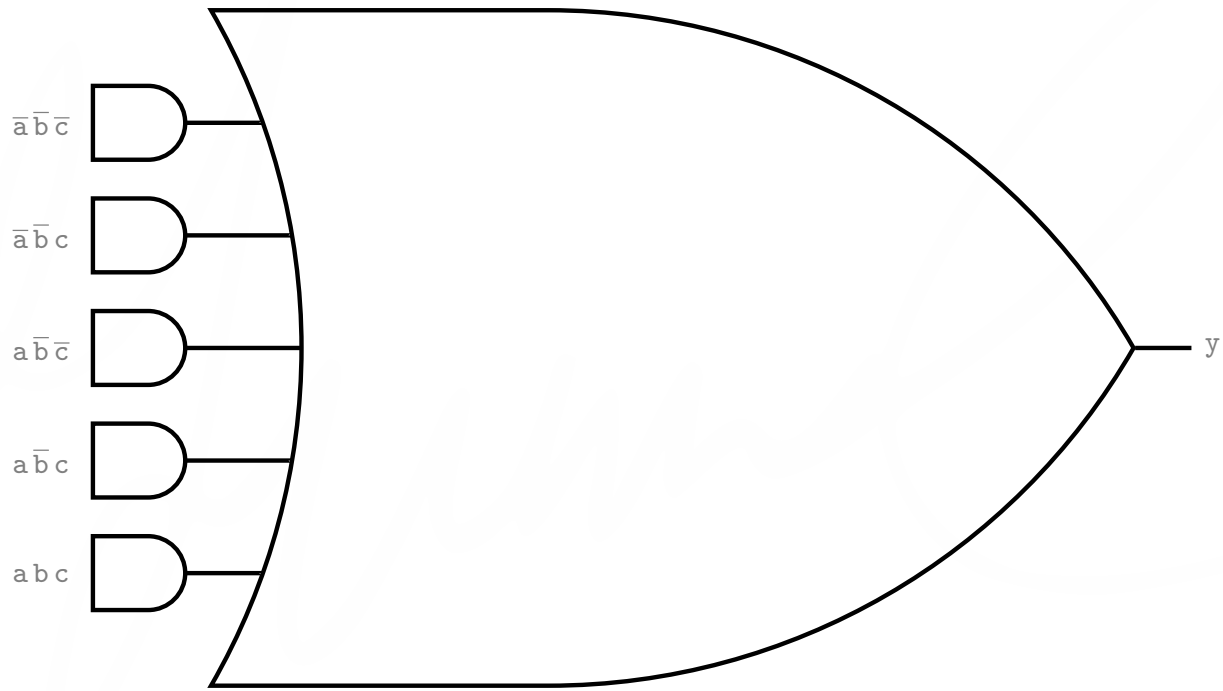
$$y = \overline{a}\,\overline{b}\,\overline{c} + \overline{a}\,\overline{b}\,c + a\,\overline{b}\,\overline{c} + a\,\overline{b}\,c + a\,b\,c$$

becomes

$$y = M_2 M_3 M_6$$

(3) gates

translate the expression into a circuit

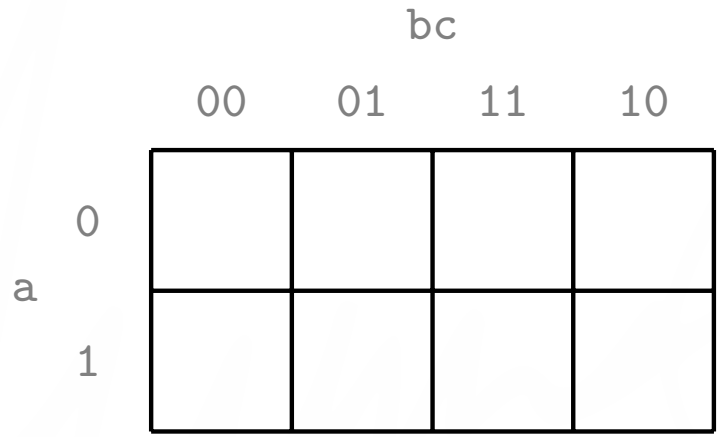


This approach works, but it is rarely the smallest circuit. Fewer terms means fewer gates, less space, and less delay.

We want a systematic way to find a small expression.

Karnaugh maps

A **Karnaugh map** rewrites the truth table as a grid. Neighbouring cells differ in exactly one input, so their column labels follow a special order.



the columns step 00, 01, 11, 10, changing one bit at a time

We fill each cell with the output for that input.

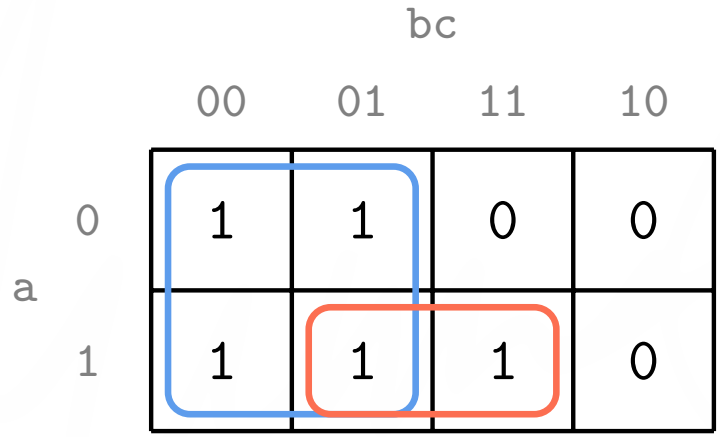
a \ bc	00	01	11	10
0	1	1	0	0
1	1	1	1	0

We cover the 1s (when using sum-of-products), or 0s (when using product-of-sums) with as few groups as possible.

A valid group obeys three rules:

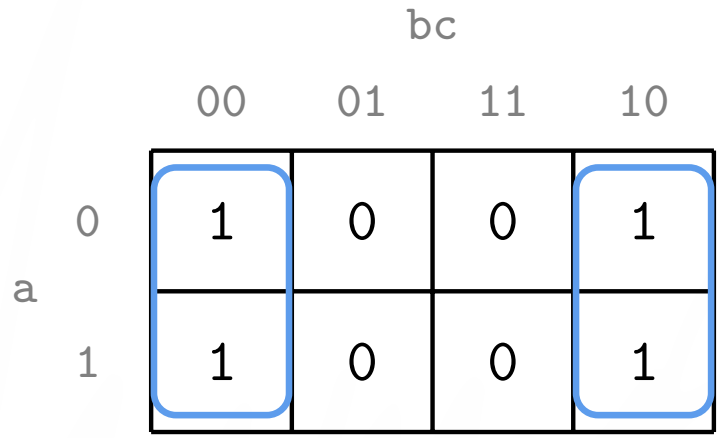
- it is a rectangle of 1s (or 0s)
- its size is a power of two
- it may overlap other groups and may wrap around the edges

We circle the largest groups of adjacent 1s (when using a sum-of-products) or 0s (when using product-of-sums) whose size is a power of two. Each group becomes one term.



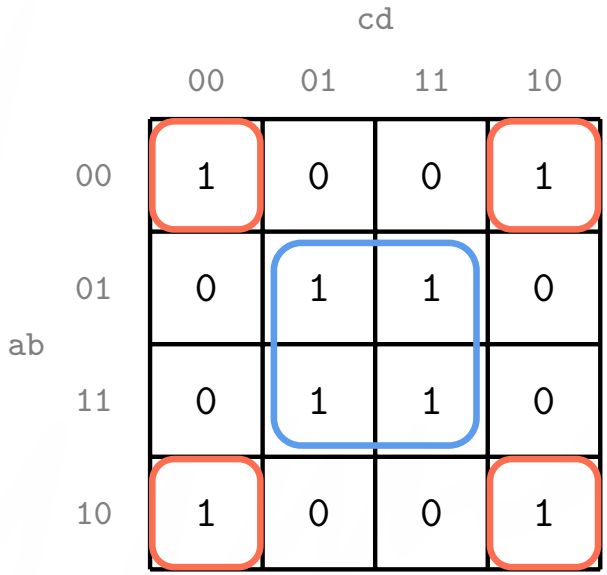
$$y = \overline{b} + ac$$

A group may also wrap around the edges of the map.



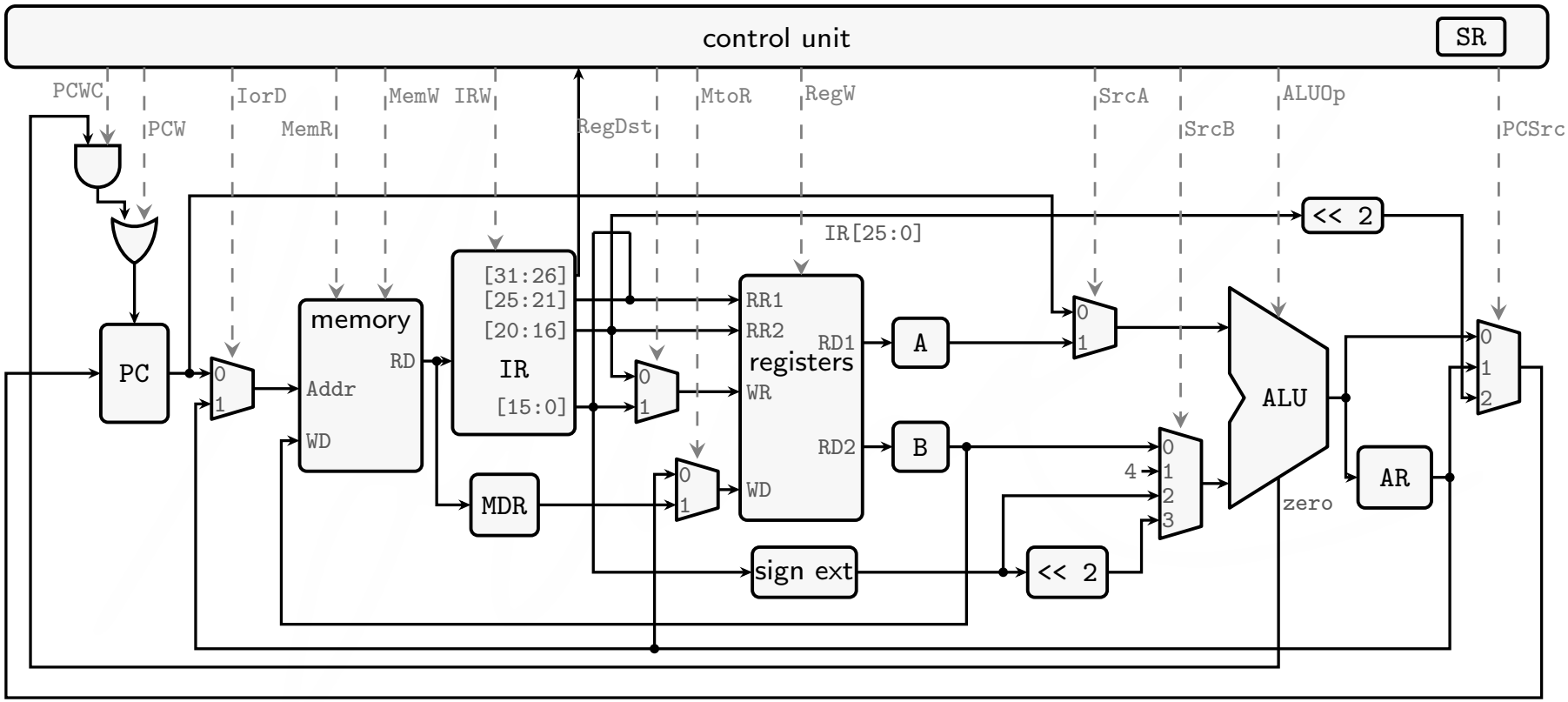
the leftmost and rightmost columns are neighbours, so together they count as a single group

Larger maps work the same way.



the four corners are all neighbours here, wrapping both the rows and the columns

We now have all the knowledge we need to build a general-purpose **computer**.



The first step is to learn how to perform **binary arithmetic**.